

Lab 10: Test MongoDB, Ollama and the sandbox

Notebook for running inside the `comp40771` container.

It demonstrates how to:

- configure the MongoDB connection
- create collections and indexes
- save data
- load data
- update data
- delete data
- verify if ollama is running
- list the models
- runs an ollama query
- tests the sandbox container

The notebook assumes MongoDB is reachable through Docker Compose at `mongodb:27017`.

1. Install dependencies

Run the next cell only if `pymongo` is not already available in the container.

```
# Uncomment if needed
!pip install pymongo==4.10.1

Collecting pymongo==4.10.1
  Downloading pymongo-4.10.1-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (22 kB)
Collecting dnspython<3.0.0,>=1.16.0 (from pymongo==4.10.1)
  Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Downloading pymongo-4.10.1-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.9 MB)
----- 1.9/1.9 MB 9.0 MB/s 0:00:00m
eta 0:00:01
ongo
----- 2/2 [pymongo]m1/2 [pymongo]
ongo-4.10.1
```

2. Configuration

```
import os
from pprint import pprint
from pymongo import MongoClient
```

```
MONGO_URI = os.getenv('MONGO_URI', 'mongodb://mongodb:27017')
MONGO_DB = os.getenv('MONGO_DB', 'lab10_audit')
```

```
print('Mongo URI:', MONGO_URI)
print('Database:', MONGO_DB)
```

```
Mongo URI: mongodb://mongodb:27017
Database: lab10_audit
```

3. Connect to MongoDB

```
client = MongoClient(MONGO_URI)
db = client[MONGO_DB]
```

```
print('Connected successfully')
print('Collections currently present:', db.list_collection_names())
```

```
Connected successfully
Collections currently present: ['sandbox_classifications',
'agent_observations', 'sandbox_runs', 'reports', 'telemetry_logs']
```

4. Create collections and indexes

```
required_collections = [
    'sandbox_runs',
    'telemetry_logs',
    'sandbox_classifications',
    'reports'
]
```

```
existing = set(db.list_collection_names())
for name in required_collections:
    if name not in existing:
        db.create_collection(name)
```

```
db['sandbox_runs'].create_index('run_id', unique=True)
db['telemetry_logs'].create_index('run_id')
db['sandbox_classifications'].create_index('run_id')
db['reports'].create_index('run_id')
```

```
print('Collections after setup:', db.list_collection_names())
```

```
Collections after setup: ['sandbox_classifications',
'agent_observations', 'sandbox_runs', 'reports', 'telemetry_logs']
```

5. Create / Save data

```
run_document = {
    'run_id': 'run_001',
```

```

        'api_id': 'api07',
        'started_at': '2026-03-15T10:00:00Z',
        'status': 'completed'
    }

    telemetry_document = {
        'run_id': 'run_001',
        'cpu_peak': 78.5,
        'network_attempts': 3,
        'file_changes': 2,
        'thread_count': 5
    }

    classification_document = {
        'run_id': 'run_001',
        'api_id': 'api07',
        'classification': 'suspicious',
        'confidence': 0.81,
        'explanation': 'Repeated outbound connections combined with file
writes.'
    }

    report_document = {
        'run_id': 'run_001',
        'api_id': 'api07',
        'risk_score': 0.76,
        'classification': 'suspicious',
        'recommended_mitigation': 'Restrict network access and filesystem
write scope.'
    }

# Upsert avoids duplicate key errors if the notebook is run more than
once
    db['sandbox_runs'].update_one({'run_id': run_document['run_id']},
    {'$set': run_document}, upsert=True)
    db['telemetry_logs'].update_one({'run_id':
telemetry_document['run_id']}, {'$set': telemetry_document},
upsert=True)
    db['sandbox_classifications'].update_one({'run_id':
classification_document['run_id']}, {'$set': classification_document},
upsert=True)
    db['reports'].update_one({'run_id': report_document['run_id']},
    {'$set': report_document}, upsert=True)

    print('Documents saved successfully')

```

Documents saved successfully

6. Load / Read data

```
run = db['sandbox_runs'].find_one({'run_id': 'run_001'})
telemetry = db['telemetry_logs'].find_one({'run_id': 'run_001'})
classification = db['sandbox_classifications'].find_one({'run_id': 'run_001'})
report = db['reports'].find_one({'run_id': 'run_001'})
```

```
print('Run:')
pprint(run)
print('\nTelemetry:')
pprint(telemetry)
print('\nClassification:')
pprint(classification)
print('\nReport:')
pprint(report)
```

Run:

```
{'_id': ObjectId('69b976140b94c7345f05ffe4'),
 'api_id': 'api07',
 'run_id': 'run_001',
 'started_at': '2026-03-15T10:00:00Z',
 'status': 'completed'}
```

Telemetry:

```
{'_id': ObjectId('69b976140b94c7345f05ffe6'),
 'cpu_peak': 78.5,
 'file_changes': 2,
 'network_attempts': 3,
 'run_id': 'run_001',
 'thread_count': 5}
```

Classification:

```
{'_id': ObjectId('69b976140b94c7345f05ffe8'),
 'api_id': 'api07',
 'classification': 'suspicious',
 'confidence': 0.81,
 'explanation': 'Repeated outbound connections combined with file writes.',
 'run_id': 'run_001'}
```

Report:

```
{'_id': ObjectId('69b976140b94c7345f05ffea'),
 'api_id': 'api07',
 'classification': 'suspicious',
 'recommended_mitigation': 'Restrict network access and filesystem write',
 'scope.',
 'risk_score': 0.76,
 'run_id': 'run_001'}
```

7. Query multiple documents

```
results = list(db['sandbox_classifications'].find({
    'classification': {'$in': ['suspicious', 'potentially_harmful']}
}))

print(f'Found {len(results)} suspicious or harmful classifications')
for item in results:
    pprint(item)

Found 1 suspicious or harmful classifications
{'_id': ObjectId('69b976140b94c7345f05ffe8'),
 'api_id': 'api07',
 'classification': 'suspicious',
 'confidence': 0.81,
 'explanation': 'Repeated outbound connections combined with file
writes.',
 'run_id': 'run_001'}
```

8. Update data

```
update_result = db['sandbox_classifications'].update_one(
    {'run_id': 'run_001'},
    {'$set': {
        'classification': 'potentially_harmful',
        'confidence': 0.93,
        'explanation': 'High-risk behaviour confirmed after additional
telemetry review.'
    }}
)

print('Modified count:', update_result.modified_count)
pprint(db['sandbox_classifications'].find_one({'run_id': 'run_001'}))

Modified count: 1
{'_id': ObjectId('69b976140b94c7345f05ffe8'),
 'api_id': 'api07',
 'classification': 'potentially_harmful',
 'confidence': 0.93,
 'explanation': 'High-risk behaviour confirmed after additional
telemetry '
               'review.',
 'run_id': 'run_001'}
```

9. Update many documents

```
update_many_result = db['sandbox_runs'].update_many(
    {'status': 'completed'},
    {'$set': {'reviewed': True}}
)
```

```
print('Modified count:', update_many_result.modified_count)
```

Modified count: 1

10. Delete data

```
# Delete one example report
delete_one_result = db['reports'].delete_one({'run_id': 'run_001'})
print('Deleted from reports:', delete_one_result.deleted_count)
```

```
# Delete many benign classifications if any exist
delete_many_result =
db['sandbox_classifications'].delete_many({'classification':
'benign'})
print('Deleted benign classifications:',
delete_many_result.deleted_count)
```

Deleted from reports: 1
Deleted benign classifications: 0

11. Verify final state

```
for collection_name in required_collections:
    count = db[collection_name].count_documents({})
    print(f'{collection_name}: {count} document(s)')

sandbox_runs: 1 document(s)
telemetry_logs: 1 document(s)
sandbox_classifications: 1 document(s)
reports: 0 document(s)
```

12. Close the connection

```
client.close()
print('MongoDB connection closed')
```

MongoDB connection closed

13. Test ollama

```
import requests
import json

OLLAMA_BASE = "http://ollama:11434"

print("Ollama base URL:", OLLAMA_BASE)

Ollama base URL: http://ollama:11434
```

14. Check if Ollama server is reachable

```
try:
    r = requests.get(f"{OLLAMA_BASE}/")
    print("Status:", r.status_code)
    print("Response:", r.text[:200])
except Exception as e:
    print("Connection failed:", e)
```

Connection failed: HTTPConnectionPool(host='ollama', port=11434): Max retries exceeded with url: / (Caused by NameResolutionError("HTTPConnection(host='ollama', port=11434): Failed to resolve 'ollama' ([Errno -5] No address associated with hostname)))

15. List installed models

```
r = requests.get(f"{OLLAMA_BASE}/api/tags")
models = r.json()

print(json.dumps(models, indent=2))
```

```
-----
-----
gaierror                                Traceback (most recent call
last)
File ~/.env/lib/python3.12/site-packages/urllib3/connection.py:204, in
HTTPConnection._new_conn(self)
    203 try:
--> 204     sock = connection.create_connection(
    205         (self._dns_host, self.port),
    206         self.timeout,
    207         source_address=self.source_address,
    208         socket_options=self.socket_options,
    209     )
    210 except socket.gaierror as e:

File
~/.env/lib/python3.12/site-packages/urllib3/util/connection.py:60, in
create_connection(address, timeout, source_address, socket_options)
    58     raise LocationParseError(f"'{host}', label empty or too
long") from None
--> 60 for res in socket.getaddrinfo(host, port, family,
socket.SOCK_STREAM):
    61     af, socktype, proto, canonname, sa = res

File /usr/lib/python3.12/socket.py:963, in getaddrinfo(host, port,
family, type, proto, flags)
    962 addrlist = []
--> 963 for res in _socket.getaddrinfo(host, port, family, type,
```

```
proto, flags):
    964         af, socktype, proto, canonname, sa = res
```

gaierror: [Errno -5] No address associated with hostname

The above exception was the direct cause of the following exception:

NameResolutionError Traceback (most recent call last)

File

```
~/env/lib/python3.12/site-packages/urllib3/connectionpool.py:787, in
HTTPConnectionPool.urlopen(self, method, url, body, headers, retries,
redirect, assert_same_host, timeout, pool_timeout, release_conn,
chunked, body_pos, preload_content, decode_content, **response_kw)
    786 # Make the request on the HTTPConnection object
```

```
--> 787 response = self._make_request(
    788     conn,
    789     method,
    790     url,
    791     timeout=timeout_obj,
    792     body=body,
    793     headers=headers,
    794     chunked=chunked,
    795     retries=retries,
    796     response_conn=response_conn,
    797     preload_content=preload_content,
    798     decode_content=decode_content,
    799     **response_kw,
    800 )
    802 # Everything went great!
```

File

```
~/env/lib/python3.12/site-packages/urllib3/connectionpool.py:493, in
HTTPConnectionPool._make_request(self, conn, method, url, body,
headers, retries, timeout, chunked, response_conn, preload_content,
decode_content, enforce_content_length)
```

```
    492 try:
--> 493     conn.request(
    494         method,
    495         url,
    496         body=body,
    497         headers=headers,
    498         chunked=chunked,
    499         preload_content=preload_content,
    500         decode_content=decode_content,
    501         enforce_content_length=enforce_content_length,
    502     )
    504 # We are swallowing BrokenPipeError (errno.EPIPE) since the
server is
    505 # legitimately able to close the connection after sending a
```



```
506 # With this behaviour, the received response is still
readable.
```

```
File /usr/lib/python3.12/http/client.py:1351, in
HTTPConnection.endheaders(self, message_body, encode_chunked)
    1350     raise CannotSendHeader()
-> 1351 self. send output(message body, encode chunked=encode chunked)
```

```
File /usr/lib/python3.12/http/client.py:1055, in
HTTPConnection.send(self, data)
    1054 if self.auto_open:
-> 1055     self.connect()
    1056 else:
```

```
File ~/.env/lib/python3.12/site-packages/urllib3/connection.py:211, in HTTPConnection._new_conn(self)
    210 except socket.gaierror as e:
--> 211     raise NameResolutionError(self.host, self, e) from e
    212 except SocketTimeout as e:
```

The above exception was the direct cause of the following exception:

MaxRetryError Traceback (most recent call

```

last)
File ~/.env/lib/python3.12/site-packages/requests/adapters.py:644, in
HTTPAdapter.send(self, request, stream, timeout, verify, cert,
proxies)
    643 try:
--> 644     resp = conn.urlopen(
    645         method=request.method,
    646         url=url,
    647         body=request.body,
    648         headers=request.headers,
    649         redirect=False,
    650         assert_same_host=False,
    651         preload_content=False,
    652         decode_content=False,
    653         retries=self.max_retries,
    654         timeout=timeout,
    655         chunked=chunked,
    656     )
    658 except (ProtocolError, OSError) as err:

```

```

File
~/.env/lib/python3.12/site-packages/urllib3/connectionpool.py:841, in
HTTPConnectionPool.urlopen(self, method, url, body, headers, retries,
redirect, assert_same_host, timeout, pool_timeout, release_conn,
chunked, body_pos, preload_content, decode_content, **response_kw)
    839     new_e = ProtocolError("Connection aborted.", new_e)
--> 841     retries = retries.increment(
    842         method, url, error=new_e, _pool=self,
    _stacktrace=sys.exc_info()[2]
    843     )
    844     retries.sleep()

```

```

File ~/.env/lib/python3.12/site-packages/urllib3/util/retry.py:535, in
Retry.increment(self, method, url, response, error, _pool,
_stacktrace)
    534     reason = error or ResponseError(cause)
--> 535     raise MaxRetryError(_pool, url, reason) from reason #
type: ignore[arg-type]
    537     log.debug("Incremented Retry for (url='%s'): %r", url,
new_retry)

```

MaxRetryError: HTTPConnectionPool(host='ollama', port=11434): Max retries exceeded with url: /api/tags (Caused by NameResolutionError("HTTPConnection(host='ollama', port=11434): Failed to resolve 'ollama' ([Errno -5] No address associated with hostname)))

During handling of the above exception, another exception occurred:

ConnectionError Traceback (most recent call

```

last)
Cell In[15], line 1
----> 1 r = requests.get(f"{OLLAMA_BASE}/api/tags")
      2 models = r.json()
      4 print(json.dumps(models, indent=2))

File ~/.env/lib/python3.12/site-packages/requests/api.py:73, in
get(url, params, **kwargs)
    62 def get(url, params=None, **kwargs):
    63     r"""Sends a GET request.
    64
    65     :param url: URL for the new :class:`Request` object.
    66     :param params: (optional) Dictionary of URL parameters to send
    67     :param data: (optional) Dictionary, list of tuples, bytes, or file-like
    68     :param json: (optional) Dictionary of JSON-serializable objects to
    69     :param headers: (optional) Dictionary of HTTP headers to send with the
    70     :param cookies: (optional) Dictionary of cookies to send with the
    71     :param auth: (optional) Auth tuple to enable Basic/Digest authentication
    72     :param timeout: (optional) Connect timeout in seconds.
    73     :param allow_redirects: (optional) Boolean. Enable/disable GET
    74     :param proxies: (optional) Dictionary mapping protocol to the
    75     :param verify: (optional) Boolean. Whether to verify SSL certificates.
    76     :param cert: (optional) if String, path to ssl client cert file (.pem).
    77     :param stream: (optional) if False, the entire content will be
    78     :param max_redirects: (optional) Maximum number of redirects to
    79     :param impersonate: (optional) Whether to impersonate a user.
    80     :param adapter: (optional) A requests.adapters.BaseAdapter instance
    81     :param kwargs: Arbitrary keyword arguments.
    82     :return: A requests.Response object.
    83     """
    84     method = 'GET'
    85     request = Request(method, url, params=params, data=data, headers=headers,
    86                       cookies=cookies, auth=auth, timeout=timeout,
    87                       allow_redirects=allow_redirects, proxies=proxies,
    88                       verify=verify, cert=cert, json=json, stream=stream,
    89                       max_redirects=max_redirects, impersonate=impersonate,
    90                       adapter=adapter, **kwargs)
    91     return session.send(request, **kwargs)

File ~/.env/lib/python3.12/site-packages/requests/api.py:59, in
request(method, url, **kwargs)
    55 # By using the 'with' statement we are sure the session is
    56 # avoid leaving sockets open which can trigger a
    57 # cases, and look like a memory leak in others.
    58 with sessions.Session() as session:
    59     return session.request(method=method, url=url, **kwargs)

File ~/.env/lib/python3.12/site-packages/requests/sessions.py:589, in
Session.request(self, method, url, params, data, headers, cookies,
files, auth, timeout, allow_redirects, proxies, hooks, stream, verify,
cert, json)
    584 send_kwargs = {
    585     "timeout": timeout,
    586     "allow_redirects": allow_redirects,
    587 }
    588 send_kwargs.update(settings)
--> 589 resp = self.send(prepare_request(self, url, params, data, headers,
    590                                   cookies, auth, timeout, allow_redirects,
    591                                   proxies, hooks, stream, verify, cert, json),
    592                    **send_kwargs)
    593 return resp

File ~/.env/lib/python3.12/site-packages/requests/sessions.py:703, in
Session.send(self, request, **kwargs)
    700 start = preferred_clock()
    702 # Send the request
--> 703 r = adapter.send(request, **kwargs)
    705 # Total elapsed time of the request (approximately)
    706 elapsed = preferred_clock() - start

File ~/.env/lib/python3.12/site-packages/requests/adapters.py:677, in
HTTPAdapter.send(self, request, stream, timeout, verify, cert,
proxies)
    673     if isinstance(e.reason, _SSLError):

```

```

674         # This branch is for urllib3 v1.22 and later.
675         raise SSLError(e, request=request)
--> 677     raise ConnectionError(e, request=request)
679 except ClosedPoolError as e:
680     raise ConnectionError(e, request=request)

```

ConnectionError: HTTPConnectionPool(host='ollama', port=11434): Max retries exceeded with url: /api/tags (Caused by NameResolutionError("HTTPConnection(host='ollama', port=11434): Failed to resolve 'ollama' ([Errno -5] No address associated with hostname)"))

16. Simple text generation test

```

payload = {
    "model": "llama3.2:1b",
    "prompt": "Write one sentence explaining MQTT.",
    "stream": False
}

response = requests.post(
    f"{OLLAMA_BASE}/api/generate",
    json=payload,
    timeout=120
)

data = response.json()
print(data["response"])

```

17. Test both models used in the lab

```

models = ["llama3.2:1b", "phi3:mini"]

for model in models:
    print("\nTesting model:", model)

    payload = {
        "model": model,
        "prompt": "Explain CI/CD in one sentence.",
        "stream": False
    }

    r = requests.post(f"{OLLAMA_BASE}/api/generate", json=payload)
    print(r.json()["response"])

```

18. Measure response latency

```

import time

```

```

model = "llama3.2:1b"

payload = {
    "model": model,
    "prompt": "List three advantages of containerisation.",
    "stream": False
}

start = time.time()

r = requests.post(f"{OLLAMA_BASE}/api/generate", json=payload)

end = time.time()

print("Latency:", round(end - start, 2), "seconds")
print(r.json()["response"])

```

Interact with the sandbox api

Starter code below provides a minimal Python client for interacting with the sandbox REST API. It assumes the sandbox container is reachable at <http://sandbox:8000>, which matches the environment variable configuration in your compose file. Example structure suitable for a notebook or a small Python module.

```

import requests
import json
import time

SANDBOX_BASE = "http://sandbox:8000"

print("Sandbox base:", SANDBOX_BASE)
print("-" * 60)

# -----
# Connectivity check
# -----

urls = [
    f"{SANDBOX_BASE}/",
    f"{SANDBOX_BASE}/docs",
    f"{SANDBOX_BASE}/openapi.json"
]

for url in urls:
    try:
        r = requests.get(url, timeout=3)
        print(url, "->", r.status_code)
    except Exception as e:
        print(url, "-> failed:", e)

```

```

print("-" * 60)

# -----
# Helper functions
# -----

def api_get(path):
    url = f"{SANDBOX_BASE}{path}"
    r = requests.get(url, timeout=10)

    if r.status_code >= 400:
        print("GET error:", r.status_code, r.text)

    try:
        return r.json()
    except Exception:
        return r.text

def api_post(path, payload=None):
    url = f"{SANDBOX_BASE}{path}"

    r = requests.post(url, json=payload or {}, timeout=10)

    if r.status_code == 409:
        print(f"{path} -> already active (409 conflict)")
        return {"status": "conflict"}

    if r.status_code >= 400:
        print("POST error:", r.status_code, r.text)

    try:
        return r.json()
    except Exception:
        return r.text

# -----
# Start telemetry
# -----

print("Starting telemetry...")
resp = api_post("/telemetry/start", {})
print(resp)

print("-" * 60)

# -----

```

```

# Run execution test
# -----

payload = {
    "api_id": "api01",
    "duration": 5
}

print("Running sandbox execution...")
resp = api_post("/execute", payload)
print(json.dumps(resp, indent=2))

print("-" * 60)

# -----
# Poll telemetry results
# -----

print("Polling telemetry status...")

end_time = time.time() + 15
while time.time() < end_time:
    try:
        data = api_get("/telemetry/status")
        print(json.dumps(data, indent=2))
    except Exception as e:
        print("Polling failed:", e)

    time.sleep(3)

print("Polling finished.")

```

Sandbox API Interaction Script -- Explanation

Code in the notebook cell demonstrates a minimal Python client for interacting with the Sandbox REST API used in the lab environment. The script performs four main tasks: verifying connectivity, defining helper functions for API communication, starting telemetry monitoring, and executing a test workload while collecting telemetry data.

1. Configuration and Imports

The script begins by importing required Python libraries:

- **requests** -- used to communicate with the REST API.
- **json** -- formats responses into readable JSON output.

- **time** -- controls polling intervals.

The variable `SANDBOX_BASE` defines the base URL of the sandbox API:

```
SANDBOX_BASE = "http://sandbox:8000"
```

Inside the Docker network, the hostname `sandbox` resolves to the sandbox container.

2. Connectivity Check

The first block verifies that the API is reachable. Three endpoints are tested:

- `/` -- basic root endpoint
- `/docs` -- Swagger documentation generated by FastAPI
- `/openapi.json` -- OpenAPI specification describing the API

Each endpoint is queried with a short timeout to confirm that the service is running.

Purpose of this step is to quickly diagnose networking issues before attempting more complex API calls.

3. REST Helper Functions

Two helper functions simplify interaction with the API.

`api_get(path)`

Function sends HTTP GET requests to retrieve information from the sandbox.

Steps performed:

1. Construct full URL from `SANDBOX_BASE` and the endpoint path.
2. Send request using `requests.get`.
3. Print errors if HTTP status ≥ 400 .
4. Attempt to parse the response as JSON.
5. If JSON parsing fails, return raw text.

This function is used for reading sandbox state such as telemetry status.

`api_post(path, payload=None)`

Function sends POST requests to trigger sandbox operations.

Steps performed:

1. Construct the endpoint URL.
2. Send POST request with optional JSON payload.
3. Detect **409 Conflict** responses.

A 409 error occurs when the requested operation is already active. For example, starting telemetry when telemetry is already running.

Instead of stopping execution, the function prints a message and continues.

4. Starting Telemetry Monitoring

Telemetry is activated with the endpoint:

```
/telemetry/start
```

Telemetry monitoring records behavioural signals such as:

- filesystem activity
- process creation
- network access
- resource usage

Information collected here can later be used for behavioural analysis and adversarial robustness testing.

5. Executing a Sandbox Task

Example workload is submitted using:

```
/execute
```

Example payload used in the script:

```
{  
  "api_id": "api01",  
  "duration": 5  
}
```

Typical meanings:

- **api_id** -- identifier of the monitored API or test scenario
- **duration** -- execution time of the monitored task

Sandbox executes the workload while telemetry continues collecting behavioural signals.

6. Polling Telemetry Status

The final section repeatedly queries:

```
/telemetry/status
```

Polling loop runs for a fixed duration and retrieves telemetry snapshots periodically.

Polling is a common monitoring strategy where the client periodically requests system state instead of maintaining a persistent connection.

7. Overall Workflow

The script implements the following pipeline:

1. Verify sandbox API availability.
2. Enable telemetry monitoring.
3. Execute a sandbox workload.
4. Poll telemetry data generated during execution.

This simple workflow forms the basis for more advanced experiments such as behavioural monitoring, anomaly detection, and adversarial robustness testing in later stages of the lab.

